

Basics of Software Concurrency

Chapter Contents

Overview	60
Concepts	61
Starter Program	61
Program Structure	62
Analysis	64
Creating and Using Tasks	65
Program Structure	65
Analysis	68
Improving Responsiveness	69
Interrupts and Event Triggering	69
Program Structure	70
Analysis	72
Reducing Task Completion Times with Finite State Machines	74
Program Structure	75
Analysis	77
Using Hardware to Save CPU Time	77
Program Structure	78
Analysis	81
Advanced Scheduling Topics	82
Waiting	82
Task Prioritization	83
Task Preemption	84
Real-Time Systems	85
Summary	86
Exercises	86

Overview

In this chapter, we explore the basic concepts of how to make a microcontroller unit (MCU) perform multiple software activities apparently simultaneously, providing overlapping (concurrent) execution. Embedded systems use peripheral hardware to perform some activities, while other activities are performed in software on the central processing unit (CPU). In order to get everything done on time, the processor's time needs to be shared among these activities. We demonstrate these processor scheduling concepts by starting with a basic program and then enhancing it to improve its **modularity**, **responsiveness**, and **CPU overhead**. Figure 3.1 presents an overview of how these topics are related.

modularity

Measure of how program is structured to group related portions and separate independent portions

responsiveness

Measure of how quickly a system responds to an input event

CPU overload

Portion of time CPU spends executing code which does not perform useful work for the application

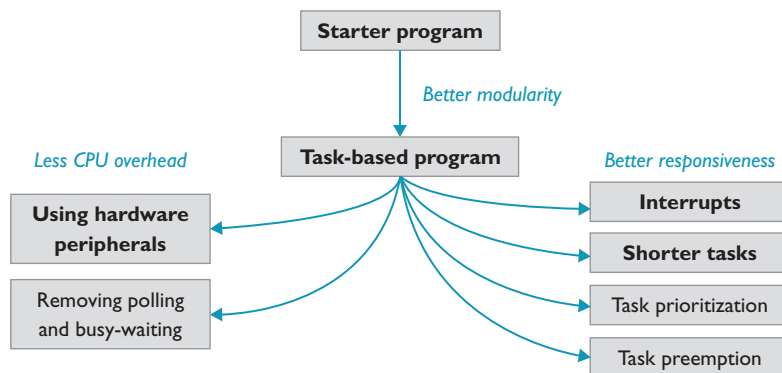


Figure 3.1 An overview of concurrency concepts presented in this chapter. Topics in bold are examined in detail.

Concepts

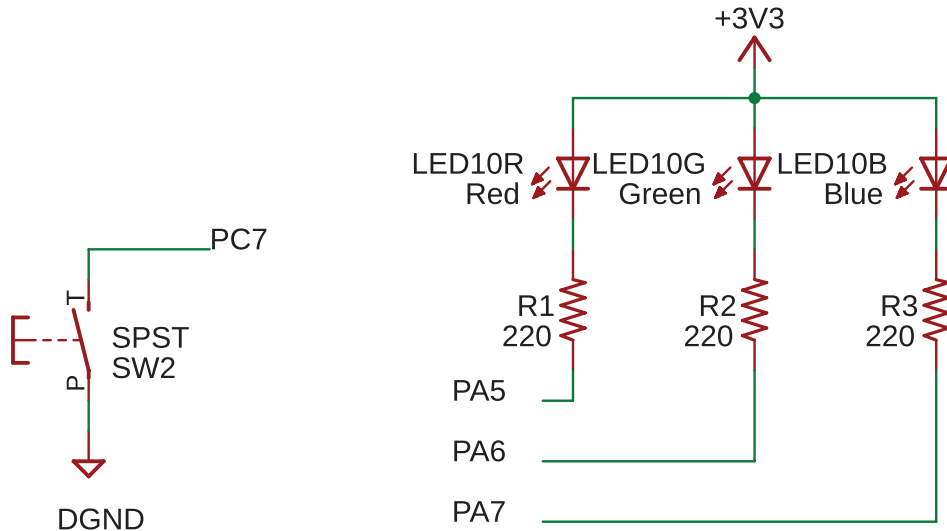


Figure 3.2 Schematic of additional switch input and LED outputs for investigating concurrency.

In order to investigate the concepts of concurrency, we will consider a system with an MCU, two switches, and an RGB (red, green, blue) LED. An RGB LED can create a wide range of colors by changing the brightness of each LED. White is created by lighting all three LED colors simultaneously. Switch 1 (SW1) is B1 on the Nucleo board, while we add switch SW2 and the RGB LED (and its current-limiting resistors R1–R3) externally as shown in Figure 3.3.

- When switch 1 is not pressed, the system displays a repeating sequence of colors (red, then green, then blue).
- When switch 1 is pressed, the system makes the LED flash white (all LEDs on) and off (all LEDs off) until the switch is released.
- When switch 2 is pressed, faster timing is used for the flashing and RGB sequences.

The time delay between the user pressing the switch and seeing the LED flash white is the system's response time for switch 1. A shorter response time is better. How we share the processor's time among the tasks is one of the main factors determining the system's responsiveness.

Starter Program

Let's start with a simple program for the LED flasher. We will put everything into one loop that reads the switches and lights the LEDs accordingly. This loop will use functions to control the LEDs (**Control_RGB_LEDs**) and delay program execution (**Delay**). It will use a macro to read the switches (**SWITCH_PRESSED**). The code is available as **ST/Code/ch3/Flasher1** in the code repository (<https://github.com/alexander-g-dean/ESF.git>).

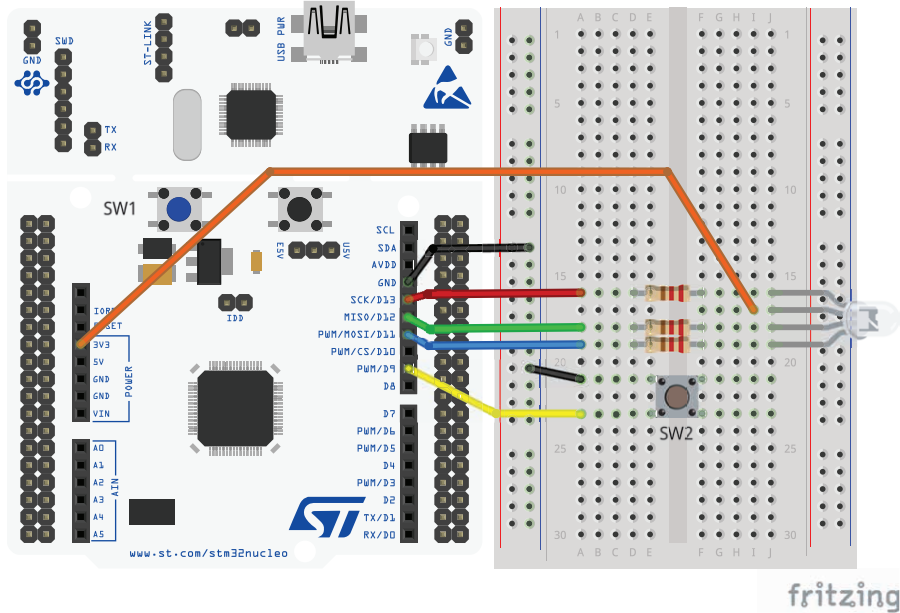


Figure 3.3 Connecting breadboard with switch, LED and resistors to Nucleo board.

Program Structure

The core of the program is shown in Listing 3.1. The program will first check to see if switch 1 is pressed. If so, the code will make the LED white, delay for some time, turn off the LED, and delay for some time again.

```
#include <stm32f091xc.h>
#include "switch.h"
#include "rgb.h"
#include "delay.h"

#define W_DELAY_SLOW      150
#define W_DELAY_FAST      70
#define RGB_DELAY_SLOW    400
#define RGB_DELAY_FAST    100

void Flasher(void) {
    uint32_t w_delay = W_DELAY_SLOW;
    uint32_t RGB_delay = RGB_DELAY_SLOW;

    Init_GPIO_RGB();
    Init_GPIO_Switches();
    while (1) {
        if (SWITCH_PRESSED(SW1_POS)) { // flash white
            Control_RGB_LEDs(1, 1, 1);
            Delay(w_delay);
            Control_RGB_LEDs(0, 0, 0);
            Delay(w_delay);
        } else { // sequence R, G, B
```

```

        Control_RGB_LEDS(1, 0, 0);
        Delay(RGB_delay);
        Control_RGB_LEDS(0, 1, 0);
        Delay(RGB_delay);
        Control_RGB_LEDS(0, 0, 1);
        Delay(RGB_delay);
    }
    if (SWITCH_PRESSED(SW2_POS)) {
        w_delay = W_DELAY_FAST;
        RGB_delay = RGB_DELAY_FAST;
    } else {
        w_delay = W_DELAY_SLOW;
        RGB_delay = RGB_DELAY_SLOW;
    }
}
}
}

```

Listing 3.1 Initial LED flasher code (in `Flasher1/main.c`).

If switch 1 is not pressed, the program will first light the red LED (turning off the others) and then wait for a fixed time. It will then light the green LED (turning off the others) and then wait for a fixed time. Finally, it will turn on the blue LED (turning off the others) and wait for a fixed time.

After flashing the LEDs, the code updates the time delays based on whether switch 2 is pressed or not. The program will then repeat.

Support code is presented in Listing 3.2 and Listing 3.3. Initialization code and header files are not included here, but are quite similar to those shown in Chapter 2 and are available in the GitHub repository.

```

#define SW_GPIO (GPIOC)    // Both switches are on GPIO C
#define SW1_POS (13)       // PC13 (User Button B1)
#define SW2_POS (7)        // PC7 (add external button)

#define MASK(x) (1UL << (x))

// Both switches are active-low, so invert data
#define SWITCH_PRESSED(x) (!((SW_GPIO)->IDR & MASK(x)))

void Init_GPIO_Switches(void);
void Init_GPIO_Switches_Interrupts(void);

```

Listing 3.2 Switch support code (in `common-driver/switch.h`) for starter program.

```

#include <stm32f091xc.h>
#include "rgb.h"
#include "field_access.h"
#include "gpio.h"

void Init_GPIO_RGB(void) {
    // Enable peripheral clock of GPIOA
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
}

```

```

// Configure the three pins as output
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER5, ESF_GPIO_MODER_OUTPUT);
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER6, ESF_GPIO_MODER_OUTPUT);
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER7, ESF_GPIO_MODER_OUTPUT);

// Turn on LEDs
GPIOA->BSRR = LED_R_ON_MSK | LED_G_ON_MSK | LED_B_ON_MSK;
}

void Control_RGB_LEDs(int r_on, int g_on, int b_on) {
    GPIOA->BSRR = (r_on) ? LED_R_ON_MSK : LED_R_OFF_MSK;
    GPIOA->BSRR = (g_on) ? LED_G_ON_MSK : LED_G_OFF_MSK;
    GPIOA->BSRR = (b_on) ? LED_B_ON_MSK : LED_B_OFF_MSK;
}

```

Listing 3.3 RGB LED support code (in `common-driver/rgb.c`) for starter program. `Control_RGB_LEDs` target LEDs in active-low configuration (output pin connected to LED cathode).

Analysis

How responsive is our system to switch 1 being pressed and released? Figure 3.4 shows that if we press the switch when the green LED is lit, the system does not start flashing until the green turns off, the blue turns on, and then turns off. The program only polls the switch between full red/green/blue color cycles, or white flash cycles.

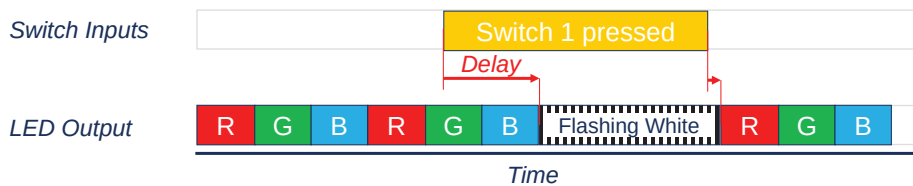


Figure 3.4 The response of an LED initial flasher program to switch press and release.

So we must hold the switch until the cycle ends. In fact, if we press the switch briefly during the color cycle and release it before the end, the program will not detect it, missing the input event. Input events shorter than the red/green/blue color cycle may be lost. The longer the code takes to run, the slower the cycle, and the greater the chance of missing an input.

Similarly, the sooner the code finishes running, the more responsive the system is. Notice in Figure 3.4 how releasing the switch results in a very short delay until the LED begins color cycling. This short delay results from the short duration of the white flash, which enables the program to check the switch more frequently.

Another disadvantage of this code is that the processor wastes quite a bit of time in its delay function. This kind of waiting is called **busy-waiting** and should be avoided except for certain special cases. We will see how later.

busy-waiting

Wasteful method of making a program wait for an event or delay. Program executes test code repeatedly in a tight loop, not sharing time with other parts of program.

Finally, the program mixes together different activities in a single function. As the program grows larger, it will be more difficult to maintain and enhance because of these interdependencies. Poorly structured code is often called *spaghetti code* because so many different parts are tangled together.

spaghetti code

Code which is poorly structured because it entangles unrelated features, complicating development and maintenance

To summarize, the system is sluggish and inefficient, may ignore brief inputs, and is structured badly.

Creating and Using Tasks

Let's restructure the code into three separate tasks, as done in project **Flasher2**. This will make it easier to develop and maintain the code. A task is a subroutine that performs a specific activity (or a closely related set of activities). Tasks simplify code development by grouping related features and processing together and separating unrelated parts. Each task has a *root function*, which may call other functions as subroutines as needed.

root function

A task's main software function, which may call other functions as subroutines

Program Structure

The first task has a root function called `Task_Read_Switches`, shown in Listing 3.4. It will determine which switches are pressed and share that information with the other tasks. We will use global variables to do this in this example, but later we will learn why global variables are dangerous and how to use other mechanisms to share information. We use the `g_` prefix in their names to indicate that these are global variables. This is helpful but not essential; it will help us and other code developers in the future by making this information obvious. Many aspects of coding style come from the desire to prevent misunderstandings and the resulting mistakes.

```
uint8_t g_flash_LED = 0;
uint32_t g_w_delay = W_DELAY_SLOW;
uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void Task_Read_Switches(void) {
    if (SWITCH_PRESSED(SW1_POS)) {
        g_flash_LED = 1; // flash white
    } else {
        g_flash_LED = 0; // RGB sequence
    }
    if (SWITCH_PRESSED(SW2_POS)) {
```